PRACTICAL 12-A

Object Oriented Programming

AIM:

To implement and understand the concepts specified in Practical 12 of the CEUE203 Object Oriented Programming practical list, using Java. The work covers Part A: Concurrent Store, Transfer, Thread Lifecycle and Part B: Thread-safe account store.

CODE SCREENSHOT:

ConcurrentStore.java

Part A > Practical 12-A > ConcurrentStore.java > ...

```java
import java.util.concurrent.*;

public class ConcurrentStore {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        ConcurrentHashMap<String, Long> balances = new ConcurrentHashMap<>();
        balances.put("AC0001", 0L);
        Thread[] ts = new Thread[10];
        for (int i = 0; i < ts.length; i++) {
            ts[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++)
                    balances.compute("AC0001", (k, v) -> v + 1);
            });
            ts[i].start();
        }
        for (Thread t : ts)
            t.join();
        System.out.println("Balance: " + balances.get("AC0001"));
    }
}

```

Code Screenshot 1

ConcurrentTransfer.java

Part A > Practical 12-A > ConcurrentTransfer.java > ...

```java
import java.util.concurrent.*;

public class ConcurrentTransfer {
    static class Bank {
        final ConcurrentHashMap<String, Long> b = new ConcurrentHashMap<>();

        synchronized boolean transfer(String from, String to, long amount) {
            if (amount <= 0 || b.get(from) < amount)
                return false;
            b.put(from, b.get(from) - amount);
            b.put(to, b.get(to) + amount);
            return true;
        }
    }

    Run | Debug
    public static void main(String[] args) {
        Bank bank = new Bank();
        bank.b.put("A", 1000L);
        bank.b.put("B", 1000L);
        System.out.println(bank.transfer(from: "A", to: "B", amount: 250));
        System.out.println(bank.b);
    }
}

```

Code Screenshot 2

ThreadLifecycle.java

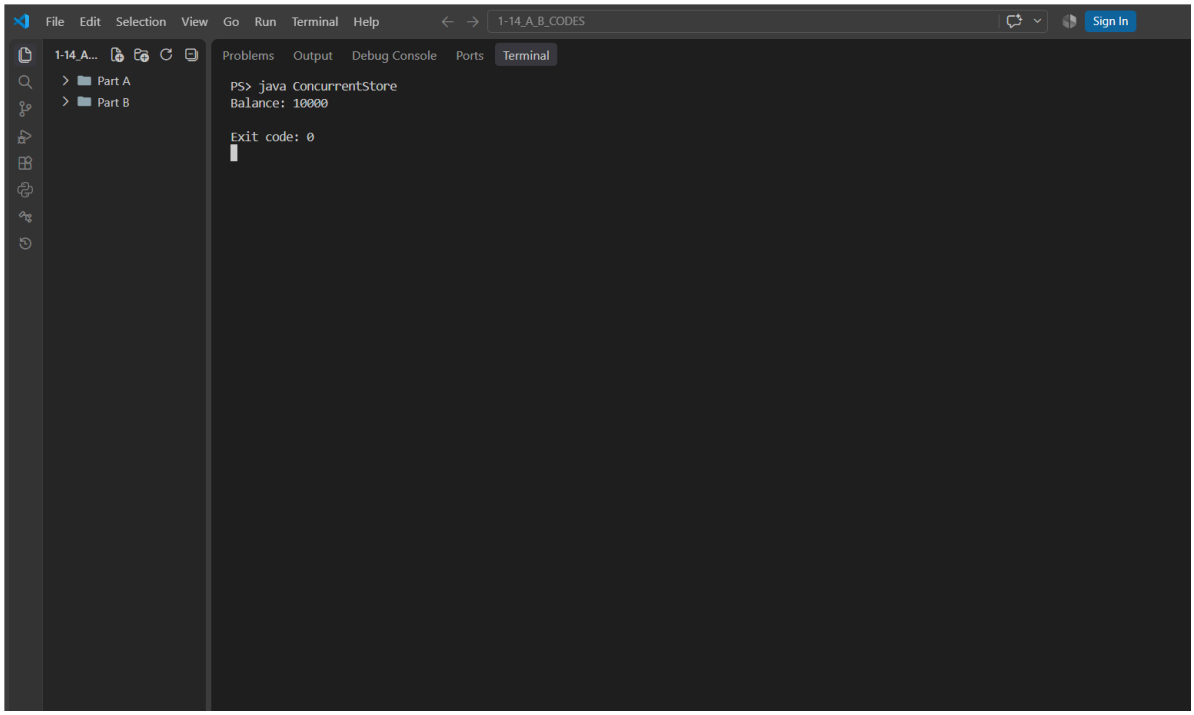
Part A > Practical 12-A > ThreadLifecycle.java > ...

```java
public class ThreadLifecycle {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> System.out.println("Running task"));
        System.out.println(t.getState());
        t.start();
        t.join();
        System.out.println(t.getState());
    }
}

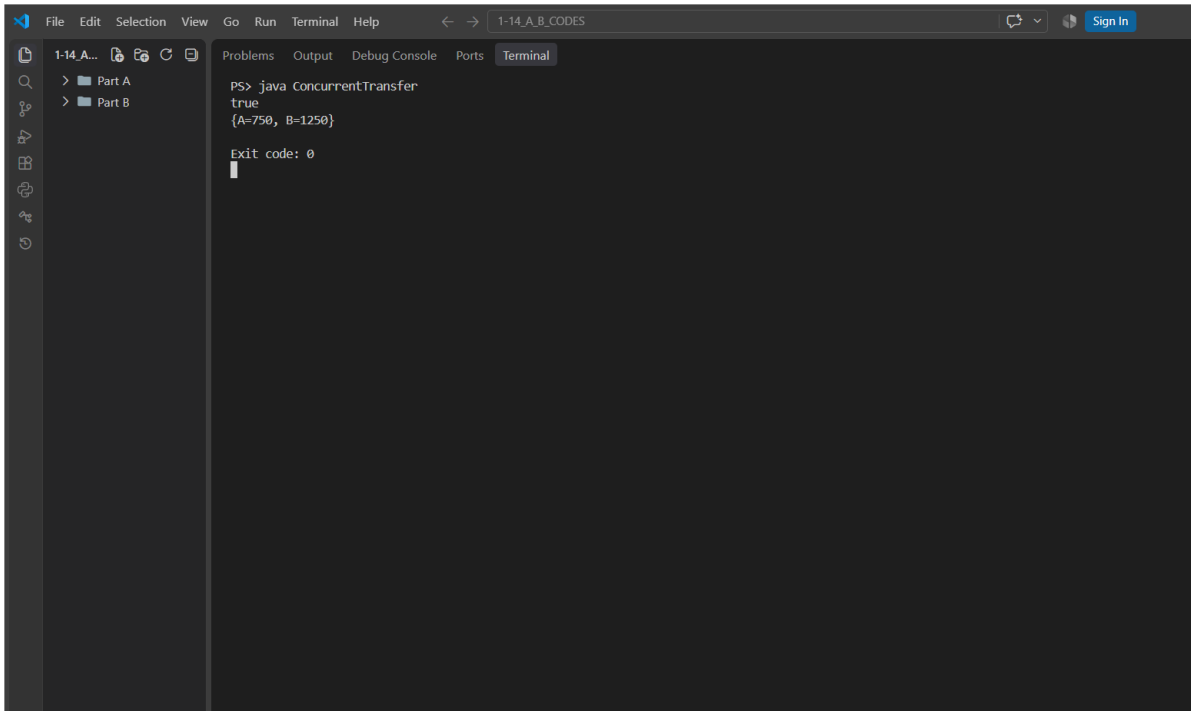
```

Code Screenshot 3

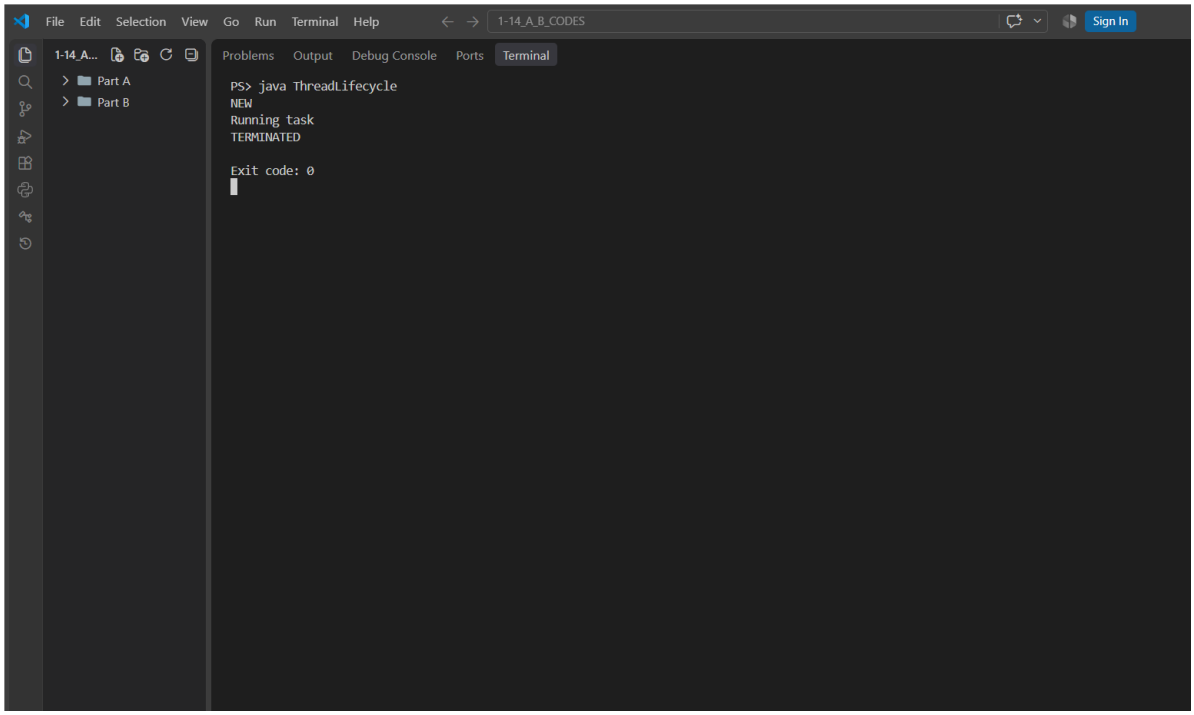
OUTPUT SCREENSHOT:



Output Screenshot 1



Output Screenshot 2



Output Screenshot 3

LEARNING OUTCOMES:

Use concurrent collections and synchronized operations to maintain consistency under concurrent access.